

ANEXO: DISEÑO DE PERSISTENCIA

Paradigma Orientado a Objetos:

Un objeto se compone de:

Estado: ¿Qué sabe o qué tiene el objeto?

Identidad: ¿Qué es? ¿Qué lo distingue de los demás objetos?

Comportamiento: ¿Qué puede hacer el objeto?

¿Qué es una clase? Es una definición de un conjunto de objetos que comparten una **estructura** común y un **comportamiento** común.

Modelo Entidad Relación:

Relación: Es la abstracción de un conjunto de asociaciones que existen entre las instancias de dos entidades. Tienen sentido bidireccional.

Cardinalidad: Indica para una instancia de una entidad A con cuantas instancias de la entidad B se relaciona.

Opcionalidad: Indica para una instancia de una entidad A, si la relación con instancias de la entidad B, es opcional u obligatoria.

Reglas de Integridad:

- 1. Integridad Relacional:** Ningún componente atributo identificador en una entidad aceptará NULOS. (ID NOT NULL)

Regla de Integridad-Clave Primaria-Base de Datos Relacional:

Un atributo “a” (posiblemente compuesto) de la relación R es una clave candidata de R, sí y sólo sí satisface las siguientes propiedades:

Unicidad: No existen dos tuplas de R con el mismo valor de “a” en un momento dado.

Minimalidad: Si “a” es un atributo compuesto, no puedo eliminar a un componente de “a” sin destruir la propiedad de unicidad.

- 2. Integridad Referencial:** Un modelo de datos no debe contener valores en sus atributos referenciales para los cuales no exista un valor concordante en el (o los) atributos identificadores en la entidad objetivo.

Problema de Impedancia:

Los tipos a ser usados en las tablas son mayormente los tipos primitivos.

La información se almacena en los objetos. Por lo tanto necesitamos transformar nuestra estructura de información de objeto a una estructura orientada a tablas.

Esto crea un fuerte acoplamiento entre la aplicación y el DBMS.

Mapeo de Clases a Bases de Datos Relacionales:

Mapeo de Clases/Objetos en Tablas:

1. Asignar una tabla para la clase.
2. Cada atributo (primitivo) se transformará en una columna en la tabla. Si el atributo es complejo (es decir, debe componerse de tipos de DBMS), o agregamos una tabla adicional para el atributo, o distribuimos el atributo en varias columnas de la tabla de la clase.
3. La columna de la clave primaria será el identificador único de la instancia, llamado identificador.
4. Cada instancia de la clase ahora será representada por una fila en esta tabla.
5. Cada asociación de conocimiento con una cardinalidad mayor que 1 se transformará en una nueva tabla. Esta nueva tabla conectará las tablas que representan los objetos que van a ser asociados.

Relaciones entre clases que requieren persistencia:

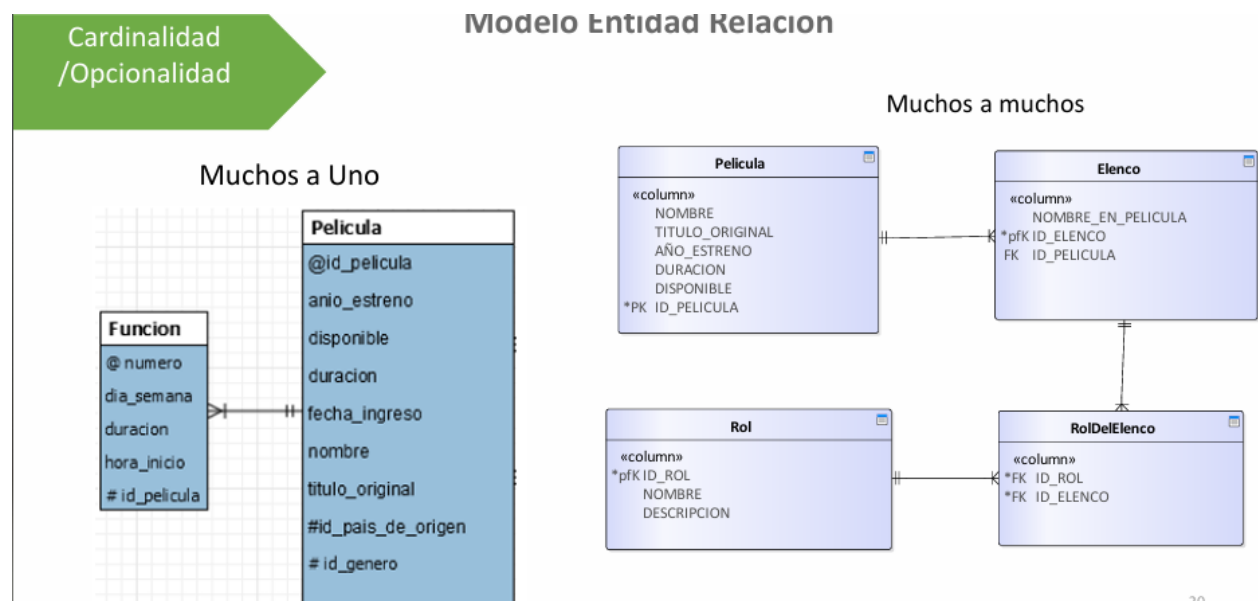
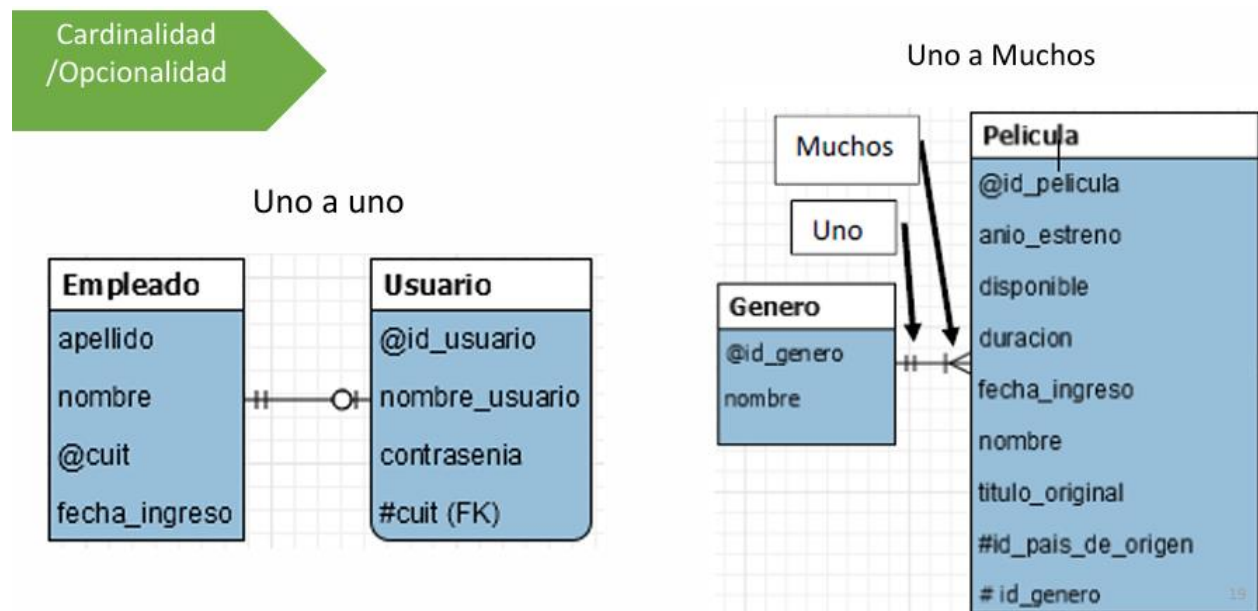
Asociación: Es una relación estructural que especifica que los objetos de un elemento se conectan a los objetos de otro.

Agregación: Es un tipo especial de asociación, que representa una relación completamente conceptual entre un “todo” y sus “partes”.

Composición: Es una variación de la agregación simple, con una fuerte relación de pertenencia y vidas coincidentes de la parte con el todo.

Generalización: Es una relación entre un elemento general (superclase o padre) y un tipo más específico de ese elemento (subclase o hijo). El hijo puede añadir nueva estructura y comportamiento, o modificar el comportamiento del padre (modifica su comportamiento heredado del padre, lo reescribe, no es que modifica el comportamiento de la superclase).

Opciones para el mapeo de Asociación:



Regla de Integridad: Relaciones

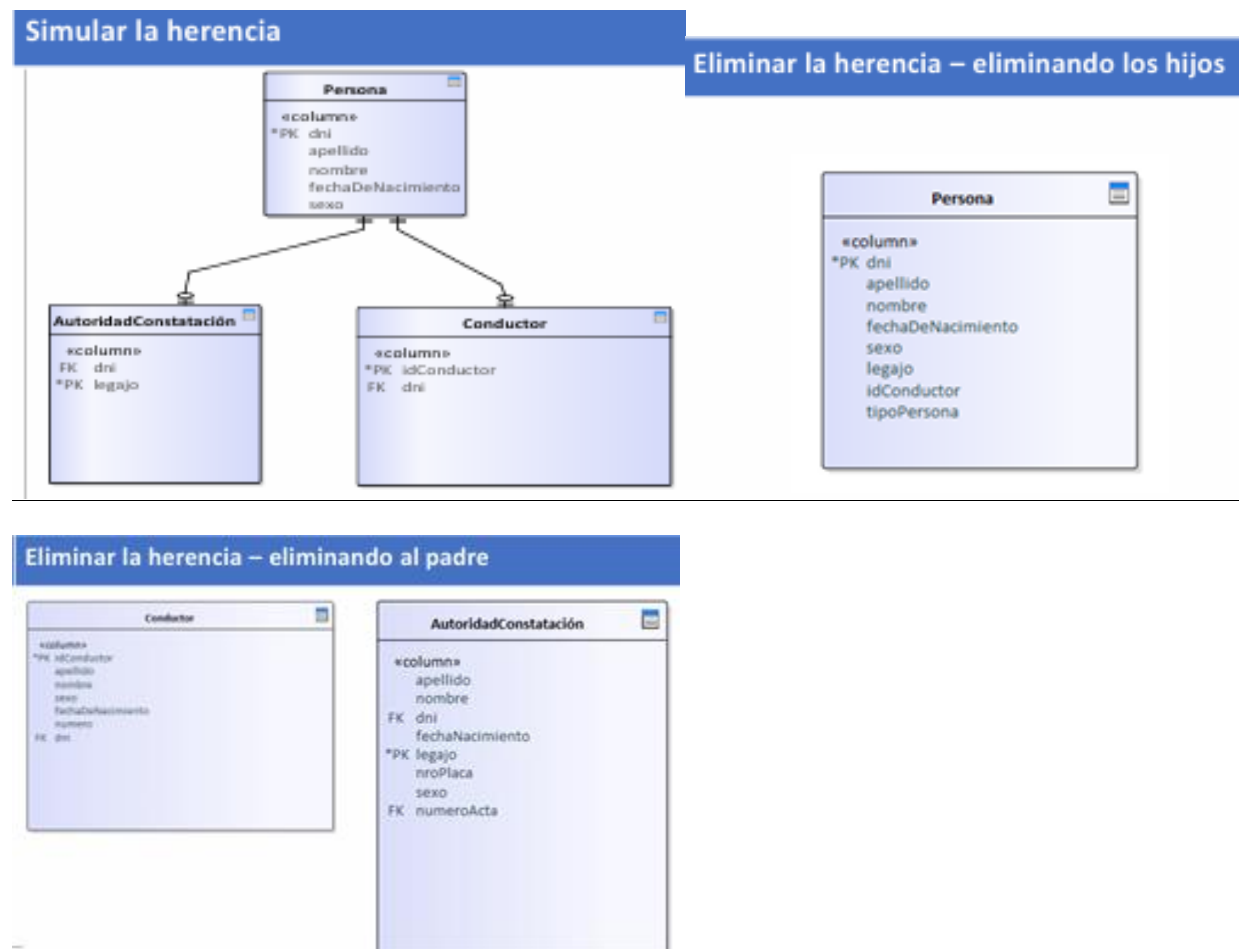
En una base de datos relacional, **nunca** registraremos información de algo que **no podamos identificar**.

- Para las claves primarias compuestas: cada valor individual de la clave primaria debe ser no nulo en su totalidad.
- Esta regla se aplica a las relaciones base únicamente.
- Se aplica sólo a las claves primarias.

Regla de Integridad: Claves Ajenas

La Foreign Key es un atributo (quizás compuesto) de una relación R2 cuyos valores deben concordar con los de la clave primaria de alguna otra relación R1.

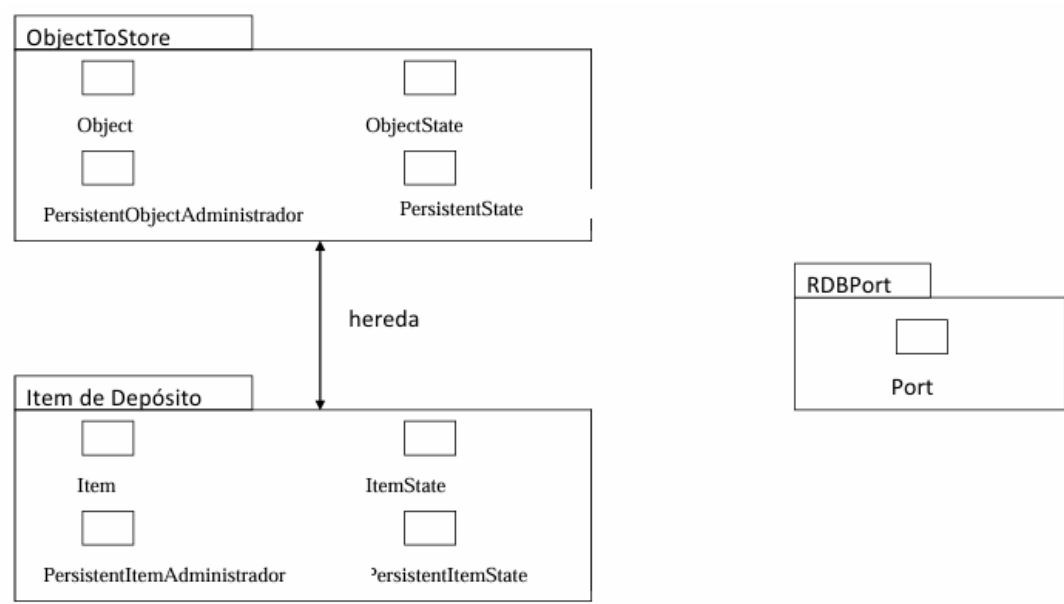
Opciones para el mapeo de la Herencia:



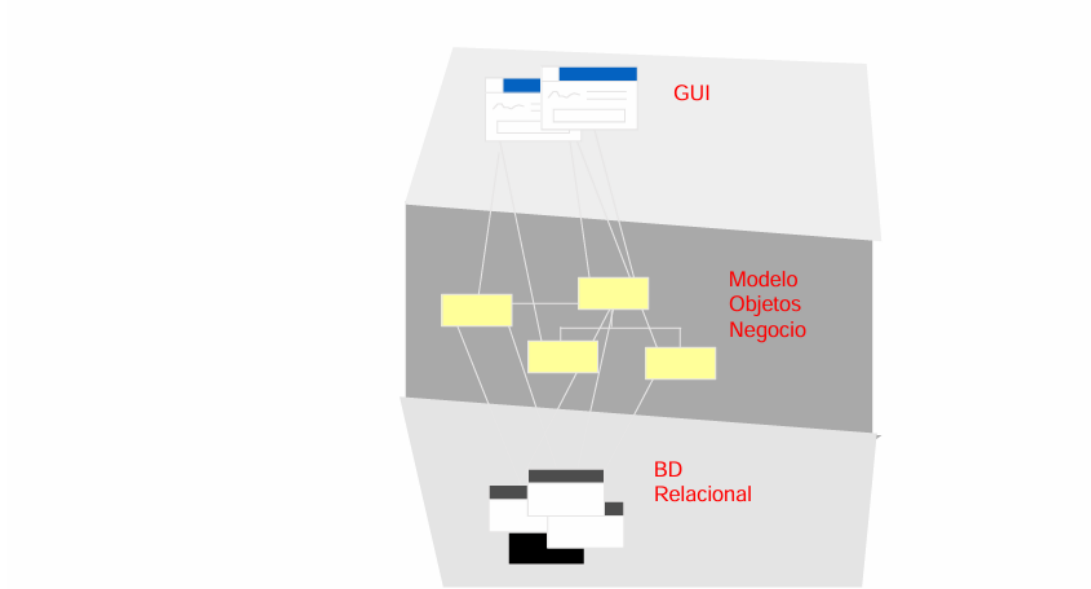
MODELOS DE PERSISTENCIA

- **Super Objeto Persistente**
- **Esquema de Persistencia**

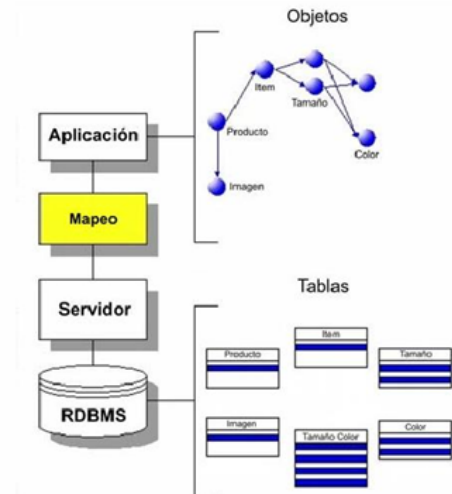
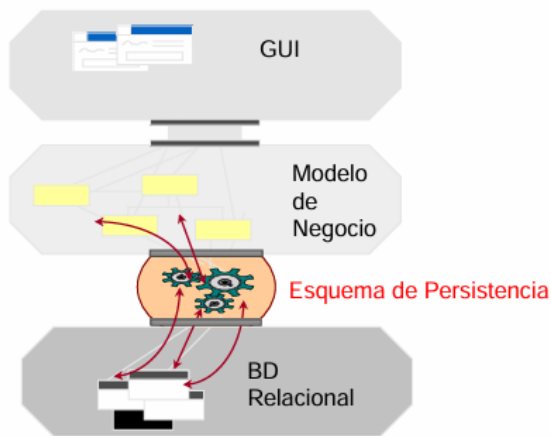
Super Objeto Persistente



SuperObjeto Persistente desde la Arquitectura



Esquema de Persistencia desde la Arquitectura



Super Objeto Persistente	Esquema de Persistencia
No crece la estructura de clases	Mantiene la cohesión entre las clases
Fácil de implementar	Soporta extensibilidad
	Mejor reusabilidad

Características de las Bases de Datos Orientadas a Objetos

- **Objetos complejos.** Debería soportar la noción de objetos complejos.
- **Identidad de Objetos.** Cada objeto debe tener una identidad independiente de sus valores internos.
- **Encapsulamiento.** Debe soportar el encapsulamiento de datos y comportamiento en los objetos.
- **Tipos o Clases.** Debería soportar un mecanismo de estructuración, en forma de tipos o clases.
- **Jerarquía.** Debería soportar la noción de herencia.
- **Ligadura tardía.** Debería soportar sobreescritura y ligadura tardía.
- **Complejidad.** El lenguaje de manipulación debería ser capaz de expresar cada función calculable.
- **Extensibilidad.** Debería ser posible agregar nuevos tipos.

Beneficios de usar OBDMS:

- Los objetos como tales pueden almacenarse en la base de datos (frecuentemente las operaciones no son almacenadas, solo están presentes en la librería de clases en la memoria primaria).
- No se necesita ninguna conversión del tipo de sistema DBMS; las clases definidas por el usuario se usan como tipos en el DBMS.
- El lenguaje del DBMS puede integrarse con un lenguaje de programación orientado a objetos. El lenguaje puede ser exactamente el mismo que se usó en la aplicación, lo que no fuerza al programador a tener dos representaciones de sus objetos.

Frameworks:

Es un conjunto extensible de clases e interfaces que colaboran para proporcionar servicios de la parte central e invariable de un subsistema lógico.

Contiene clases concretas y (especialmente) abstractas que definen las interfaces a las que ajustarse, interacciones de objetos en las que participar, y otras variantes.

Puede requerir que el usuario del framework defina subclases de las clases de este para utilizar, adaptar y extender los servicios del framework.

Tiene clases abstractas que podrían contener tanto métodos abstractos como concretos.

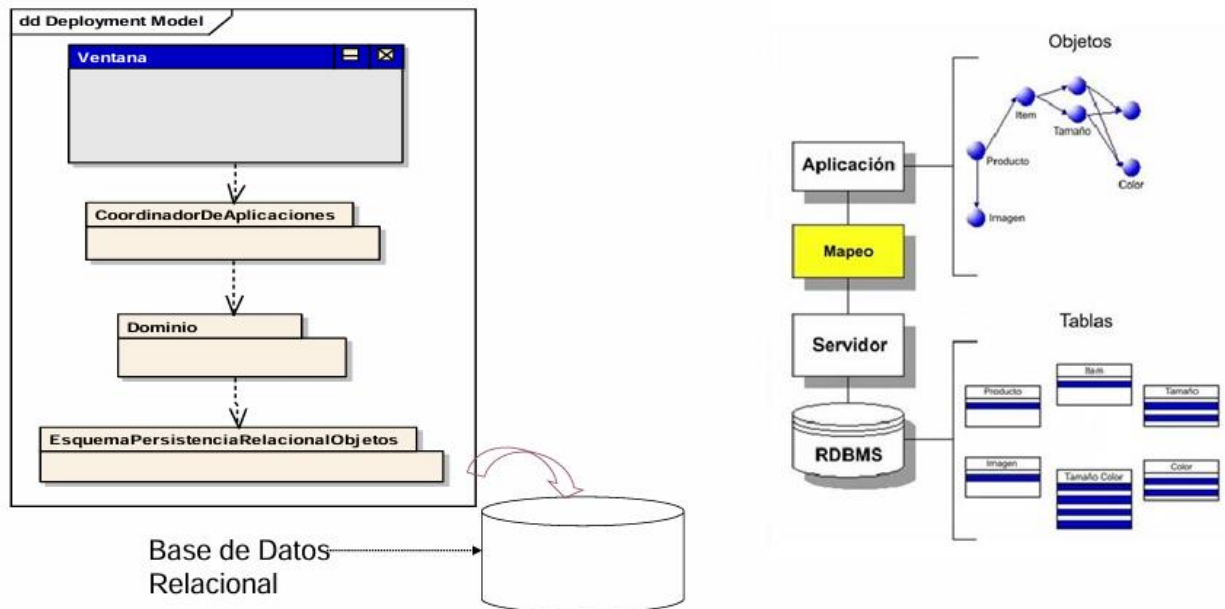
Confía en el Principio de Hollywood: “No nos llame, nosotros lo llamaremos”.

Ofrecen un alto grado de reutilización, mucho más que con las clases individuales.

Framework de Persistencia:

Debería proporcionar funciones para:

- Almacenar y recuperar objetos en un mecanismo de almacenamiento persistente.
- Confirmar y deshacer (commit y rollback) las transacciones.



Ideas a desarrollar:

- **Correspondencia (Mapping):** Entre clases y tablas; entre atributos y campos. Es decir correspondencia de “esquemas”.
- **Identidad de Objeto:** Identificador único que vincula objetos con registros.
- **Materialización:** Transformar un registro en objeto.
- **Desmaterialización:** Transformar un objeto en registro.
- **Conversor de Base de Datos:** Es el responsable de la materialización y desmaterialización de los objetos.
- **Caché:** Almacén de objetos materializados en esta memoria por cuestiones de rendimiento.
- **Estado de Transacción de los objetos:** El estado de los objetos para saber si se modificaron en función de su estado almacenado.
- **Operaciones de Transacción:** Confirmar y deshacer.
- **Materialización Perezosa:** No todos los objetos se materializan a la vez, se hace bajo demanda, es decir, cuando se lo necesita.
- **Proxies virtuales:** Referencia inteligente utilizada para la materialización perezosa.